

```
# Data handling
import numpy as np
import pandas as pd

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocessing
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Model selection
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV

# ML Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

# Evaluation
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_curve, roc_auc_score
```

```
# Read the dataset
df = pd.read_csv("Project_dataset.csv")

# Display first 5 rows
print(df.head())
```

```
      65 Female    0.7  0.1  187  16   18  6.8  3.3   0.9  1
0  62   Male  10.9  5.5  699  64  100  7.5  3.2   0.74  1
1  62   Male   7.3  4.1  490  60   68  7.0  3.3   0.89  1
2  58   Male   1.0  0.4  182  14   20  6.8  3.4   1.00  1
3  72   Male   3.9  2.0  195  27   59  7.3  2.4   0.40  1
4  46   Male   1.8  0.7  208  19   14  7.6  4.4   1.30  1
```

```
lr = LogisticRegression(class_weight='balanced')
rf = RandomForestClassifier(class_weight='balanced')
```

```
# Basic info about dataset
df.info()

# Statistical summary
df.describe()

# Check missing values
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 582 entries, 0 to 581
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0    65          582 non-null    int64
1   Female      582 non-null    object
2    0.7         582 non-null    float64
3    0.1         582 non-null    float64
4   187         582 non-null    int64
5    16         582 non-null    int64
6    18         582 non-null    int64
7    6.8         582 non-null    float64
8    3.3         582 non-null    float64
9    0.9         578 non-null    float64
10   1           582 non-null    int64
dtypes: float64(5), int64(5), object(1)
memory usage: 50.1+ KB
```

```

0
65    0
Female 0
0.7    0
0.1    0
187    0
16     0
18     0
6.8    0
3.3    0
0.9    4
1      0
```

dtype: int64

```
df = pd.read_csv("Project_dataset.csv", header=None)

df.columns = [
    "Age", "Gender", "Total_Bilirubin", "Direct_Bilirubin",
    "Alkaline_Phosphotase", "Alamine_Aminotransferase",
    "Aspartate_Aminotransferase", "Total_Protiens",
    "Albumin", "Albumin_and_Globulin_Ratio", "Dataset"
]

df.head()
```

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Aminotransferase	Total_Protiens	Albumin	Albumin_and_Globulin_Ratio	Dataset
0	65	Female	0.7	0.1	187	16	10	6.8	3.3	0.9	1
1	62	Male	10.9	5.5	699	64	10	6.8	3.3	0.9	10
2	62	Male	7.3	4.1	490	60	6	6.8	3.3	0.9	6
3	58	Male	1.0	0.4	182	14	2	6.8	3.3	0.9	2
4	72	Male	3.9	2.0	195	27	5	6.8	3.3	0.9	5

```
from sklearn.preprocessing import LabelEncoder

# Convert Gender into numbers
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])

# Fill missing values with mean
df.fillna(df.mean(), inplace=True)

# Check again
df.isnull().sum()
```

	0
Age	0
Gender	0
Total_Bilirubin	0
Direct_Bilirubin	0
Alkaline_Phosphotase	0
Alamine_Aminotransferase	0
Aspartate_Aminotransferase	0
Total_Protiens	0
Albumin	0
Albumin_and_Globulin_Ratio	0
Dataset	0

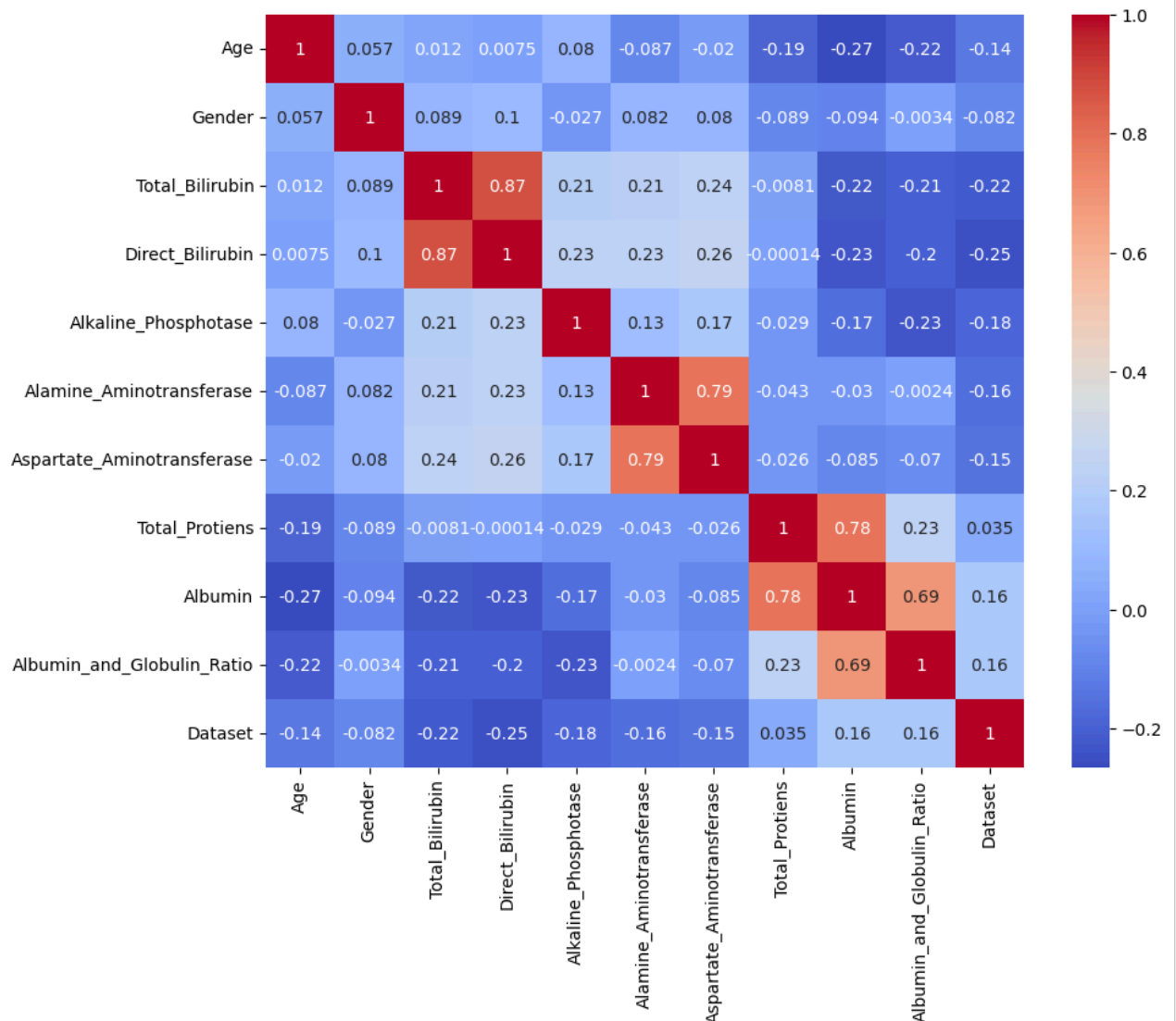
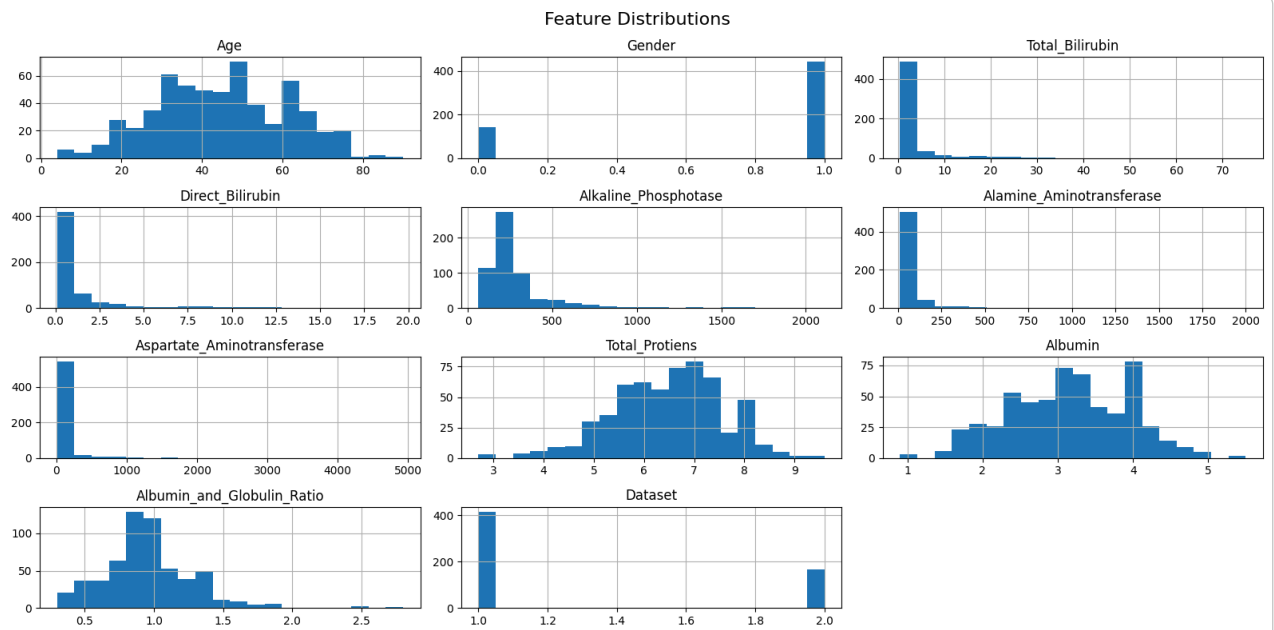
dtype: int64

df.head()

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Aminotransferase	
0	65	0	0.7	0.1	187	16		1
1	62	1	10.9	5.5	699	64		10
2	62	1	7.3	4.1	490	60		6
3	58	1	1.0	0.4	182	14		2
4	72	1	3.9	2.0	195	27		5

```
# Histogram
df.hist(figsize=(16,8), bins=20)
plt.suptitle("Feature Distributions", fontsize=16)
plt.tight_layout()
plt.show()

# Correlation Heatmap
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```



```
# Features (input)
X = df.drop("Dataset", axis=1)

# Target (output)
y = df["Dataset"]
```

```
print("X shape", X.shape)
print("y shape", y.shape)
```

```
X shape (583, 10)
y shape (583,)
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
# Create model
rf = RandomForestClassifier(class_weight='balanced')
```

```
# Train model
rf.fit(X_train, y_train)
```

```
# Predict
y_pred_rf = rf.predict(X_test)
```

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred_rf)
print("Accuracy:", accuracy)
acc_rf = accuracy_score(y_test, y_pred_rf)
prec_rf = precision_score(y_test, y_pred_rf)
rec_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)
```

```
print("Random Forest Done")
```

```
Accuracy: 0.7350427350427351
Random Forest Done
```

```
# ----- FEATURE IMPORTANCE -----#
```

```
importance = rf.feature_importances_
```

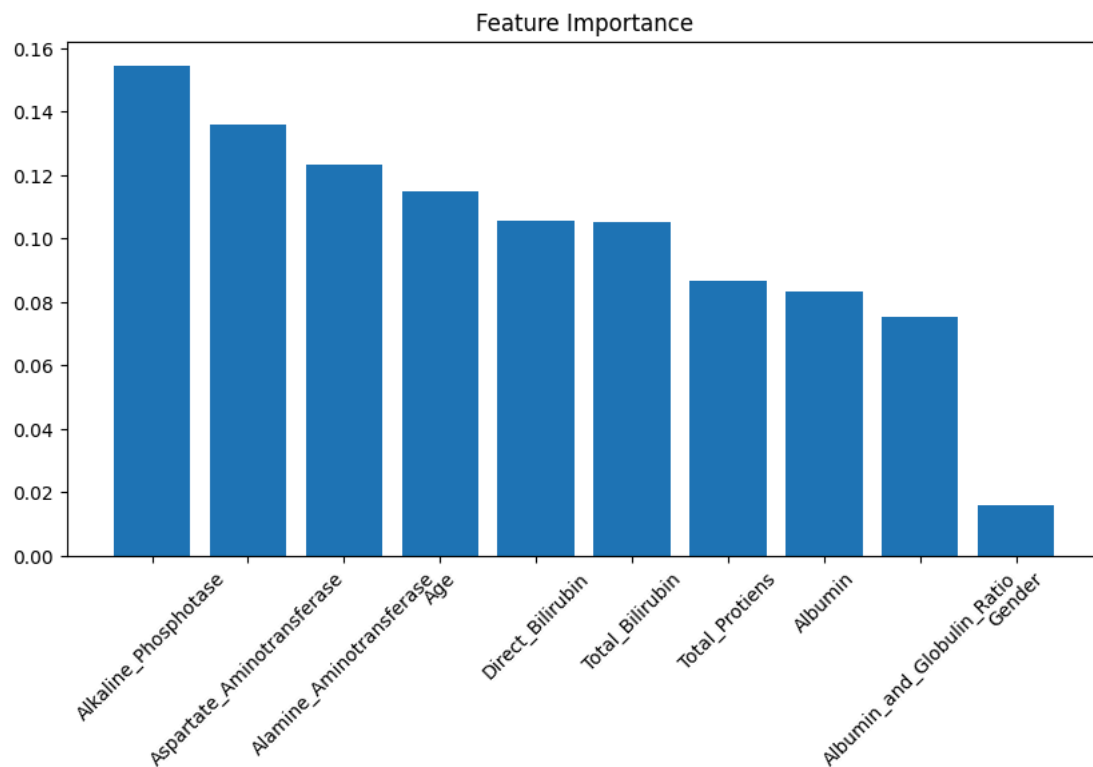
```
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance
})
```

```
feature_importance = feature_importance.sort_values(by='Importance', ascending=False)
```

```
print(feature_importance)
```

```
plt.figure(figsize=(10,5))
plt.bar(feature_importance['Feature'], feature_importance['Importance'])
plt.xticks(rotation=45)
plt.title("Feature Importance")
plt.show()
```

	Feature	Importance
4	Alkaline_Phosphotase	0.154274
6	Aspartate_Aminotransferase	0.135922
5	Alamine_Aminotransferase	0.123087
0	Age	0.114678
3	Direct_Bilirubin	0.105765
2	Total_Bilirubin	0.105059
7	Total_Protiens	0.086639
8	Albumin	0.083263
9	Albumin_and_Globulin_Ratio	0.075423
1	Gender	0.015890

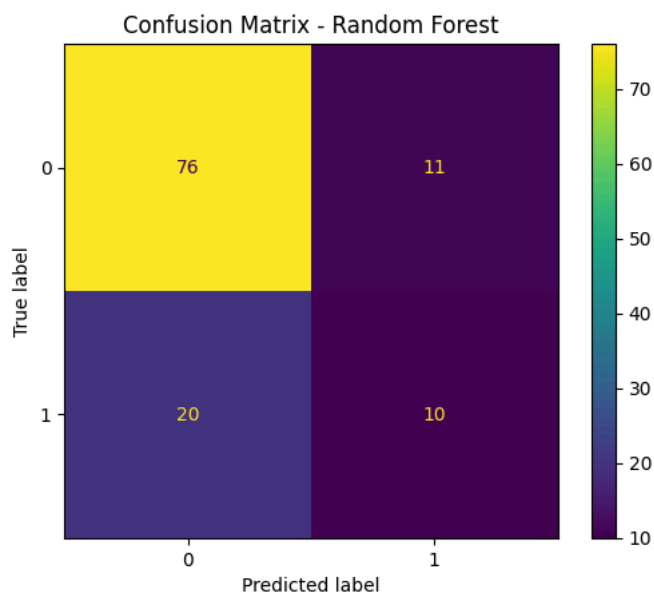


```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred_rf)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()

plt.title("Confusion Matrix - Random Forest")
plt.show()
```



```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_rf))
```

	precision	recall	f1-score	support
1	0.79	0.87	0.83	87
2	0.48	0.33	0.39	30
accuracy			0.74	117
macro avg	0.63	0.60	0.61	117
weighted avg	0.71	0.74	0.72	117

```

from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(class_weight='balanced')

lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

from sklearn.metrics import accuracy_score

print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
acc_lr = accuracy_score(y_test, y_pred_lr)
prec_lr = precision_score(y_test, y_pred_lr)
rec_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

```

Logistic Regression Accuracy: 0.6581196581196581

```

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_lr))

```

	precision	recall	f1-score	support
1	0.98	0.55	0.71	87
2	0.43	0.97	0.59	30
accuracy			0.66	117
macro avg	0.70	0.76	0.65	117
weighted avg	0.84	0.66	0.68	117

```

from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(class_weight='balanced')
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

acc_dt = accuracy_score(y_test, y_pred_dt)
prec_dt = precision_score(y_test, y_pred_dt)
rec_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)
print("Decision Tree Done")

```

Decision Tree Done

```

from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
acc_knn = accuracy_score(y_test, y_pred_knn)
prec_knn = precision_score(y_test, y_pred_knn)
rec_knn = recall_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)

print("KNN Done")

```

KNN Done

```

from sklearn.svm import SVC

svm = SVC(class_weight='balanced')
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
acc_svm = accuracy_score(y_test, y_pred_svm)
prec_svm = precision_score(y_test, y_pred_svm)
rec_svm = recall_score(y_test, y_pred_svm)
f1_svm = f1_score(y_test, y_pred_svm)

print("SVM Done")

```

SVM Done

```
from sklearn.metrics import classification_report

print("Decision Tree:\n", classification_report(y_test, y_pred_dt))
print("KNN:\n", classification_report(y_test, y_pred_knn))
print("SVM:\n", classification_report(y_test, y_pred_svm))
```

Decision Tree:

	precision	recall	f1-score	support
1	0.83	0.82	0.82	87
2	0.48	0.50	0.49	30
accuracy			0.74	117
macro avg	0.65	0.66	0.66	117
weighted avg	0.74	0.74	0.74	117

KNN:

	precision	recall	f1-score	support
1	0.81	0.72	0.76	87
2	0.38	0.50	0.43	30
accuracy			0.67	117
macro avg	0.60	0.61	0.60	117
weighted avg	0.70	0.67	0.68	117

SVM:

	precision	recall	f1-score	support
1	0.94	0.51	0.66	87
2	0.39	0.90	0.54	30
accuracy			0.61	117
macro avg	0.66	0.70	0.60	117
weighted avg	0.80	0.61	0.63	117

```
results = pd.DataFrame({
    'Model': ['Logistic Regression', 'Decision Tree', 'KNN', 'SVM', 'Random Forest'],
    'Accuracy': [acc_lr, acc_dt, acc_knn, acc_svm, acc_rf],
    'Precision': [prec_lr, prec_dt, prec_knn, prec_svm, prec_rf],
    'Recall': [rec_lr, rec_dt, rec_knn, rec_svm, rec_rf],
    'F1 Score': [f1_lr, f1_dt, f1_knn, f1_svm, f1_rf]
})

print(results)
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.658120	0.979592	0.551724	0.705882
1	Decision Tree	0.735043	0.825581	0.816092	0.820809
2	KNN	0.666667	0.807692	0.724138	0.763636
3	SVM	0.606838	0.936170	0.505747	0.656716
4	Random Forest	0.735043	0.791667	0.873563	0.830601

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Correct probability for class 2
y_prob = rf.predict_proba(X_test)[:, list(rf.classes_).index(2)]

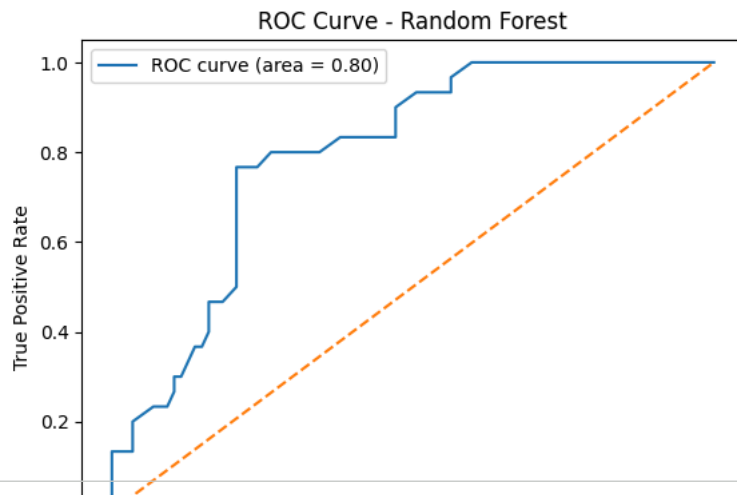
# ROC
fpr, tpr, thresholds = roc_curve(y_test, y_prob, pos_label=2)

roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], linestyle='--')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Random Forest')
plt.legend()

plt.show()
```

```
import pandas as pd
import matplotlib.pyplot as plt

importances = rf.feature_importances_
features = X.columns

importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

print(importance_df)

# Plot
plt.figure()
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.xlabel("Importance")
plt.title("Feature Importance")
```